Tribhuvan University

Institute of Engineering

PULCHOWK CAMPUS

A REPORT ON

LAB 4

Submitted To:

Department of Electronics And Computer Engineering

Pulchowk campus, IOE, TU

Submitted by:

Name: Sahas Khadka

Roll No: 080BCT071

SmartShop : Django Application

1. Project Description

SmartShop is a e-commerce web application built using the Django framework. The project demonstrates django concepts: Handling Requests & Responses, Form Data Handling & Sessions, Routing, Middleware, Templating Concepts, Database Integration , CRUD Operations, ORM Concept, Authentication & Authorization, Cookies, Sessions, Middleware for Logging, Error Handling, and Security.

2. Project Architecture: MVT (Model-View-Template)

- **Model (Database Layer):**** Manages the data and database schema. It defines the structure of the objects stored in the database.

- **View (Logical Layer):**** Contains the business logic. It processes user requests, interacts with the Models to fetch data, and returns the appropriate Template.

- **Template (Presentation Layer):**** The User Interface (UI). It defines how the data should be presented to the user using HTML and Django Template Language (DTL).

Workflow:

User → URL Dispatcher → View → Model → Template → User

3. Code Implementation of Major Components

A. Models (`store/models.py`)

Illustrates **Database Integration & ORM Concepts**.

```
from django.db import models

class Product(models.Model):

name = models.CharField(max_length=200)

price = models.DecimalField(max_digits=10, decimal_places=2)

    description = models.TextField()


    def _str_(self): return
        self.name
```

B. URL Routing (`store/urls.py`)

Illustrates **Routing & Named Routes**.

```
from django.urls import path

from . import views

urlpatterns = [

path('', views.home, name='home'),

path('products/', views.product_list, name='products'),

path('add-to-cart/<int:product_id>/', views.add_to_cart, name='add_to_cart'),

    path('cart/', views.view_cart, name='cart'),

path('checkout/', views.checkout, name='checkout'),

    path('register/', views.register_user, name='register'),

    path('login/', views.login_user, name='login'),

    path('logout/', views.logout_user, name='logout'),

]
```

C. Views (`store/views.py`)

Illustrates **Business Logic** (**Request/Response**,**Session & Cookie Handling**, **Authentication & Authorization**, **Error Handling**, **CRUD Operations(ORM)**) .

```
from django.shortcuts import render, redirect, get_object_or_404

from django.contrib.auth import login, authenticate, logout

from django.contrib.auth.forms import UserCreationForm, AuthenticationForm

from django.contrib.auth.decorators import login_required

from .models import Product

from django.contrib import messages



def home(request):

"""Simple Home View - Request & Response Handling"""

    return render(request, 'store/home.html')



def product_list(request):

"""Product Listing - Database Integration (Read)"""

    products = Product.objects.all()


cart = request.session.get('cart') if

    not cart:

cart = {}
```

```
    # Calculate total items in cart
    if cart:
        total_quantity = sum(cart.values())
    else:
        total_quantity = 0



    return render(request, 'store/products.html', {
        'products': products,
        'total_quantity': total_quantity
    })


def add_to_cart(request, product_id):
    product = get_object_or_404(Product, id=product_id)

    cart = request.session.get('cart')
    if not cart:
        cart = {}
    product_id = str(product_id)
    if product_id in cart:
        cart[product_id] += 1
    else:
        cart[product_id] = 1
```

```
    request.session['cart'] = cart
    request.session.modified = True        # Ensures session saves
```

```
    messages.success(request, f"{product.name} added to cart successfully!")


    return redirect('products')

def view_cart(request):
    """View Cart - Sessions """
    cart = request.session.get('cart', {})
    cart_items = []
    total_price = 0

    for product_id, quantity in cart.items():
        product = get_object_or_404(Product, id=int(product_id))
        item_total = product.price * quantity
        total_price += item_total
        cart_items.append({
            'product': product,
            'quantity': quantity,
            'total': item_total
        })

    return render(request, 'store/cart.html', {
        'cart_items': cart_items,
        'total_price': total_price
    })
```

```
@login_required
def checkout(request):
    """Checkout - Authorization & Sessions"""
    # Clear cart after checkout
    request.session['cart'] = {}
    return render(request, 'store/checkout.html')


def register_user(request):
    """Register - Authentication (Form Data Handling)"""
    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, "Registration successful!")
            return redirect('home')
    else:
        form = UserCreationForm()
    return render(request, 'store/register.html', {'form': form})

def login_user(request):
```

```python
    """Login - Authentication & Sessions/cookies"""
    if request.method == 'POST':
        form = AuthenticationForm(data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            # Demonstrate Cookies
            response = redirect('home')
            response.set_cookie('last_login_user', user.username)
            return response
    else:
        form = AuthenticationForm()
    return render(request, 'store/login.html', {'form': form})


def logout_user(request):
    """Logout - Sessions"""
    logout(request)
    return redirect('home')
```

D. Middleware (`store/middleware.py`)

Illustrates **Custom Middleware for Logging**.

```
import time

def RequestLoggingMiddleware(get_response):

    def middleware(request):

            # 1. Log the request info

        print(f"--- Request Logging ---")

        print(f"Path: {request.path}")

        print(f"Method: {request.method}")

        print(f"User: {request.user}")

        print(f"                                    ")


        # 2. Process request & get response

        start_time = time.time()

        response = get_response(request)

        duration = time.time() - start_time


        # 3. Log the response info

        print(f"--- Response Logging ---")

        print(f"Response Status: {response.status_code}")

        print(f"Duration: {duration:.4f}s")

        print(f"                                    ")


        return response
```

```
        return middleware
```

4. Illustration of Concepts

1. Handling Requests & Responses

Every view in `views.py` demonstrate this cycle.

```
def home(request):
"""Simple Home View - Request & Response Handling"""
    return render(request, 'store/home.html')
```

2. Form Data Handling & Sessions

A. Form Data Handling

```
def register_user(request):
"""Register - Authentication (Form Data Handling)""" if
    request.method == 'POST':
form = UserCreationForm(request.POST) if
        form.is_valid():
user = form.save() login(request, user)
messages.success(request, "Registration successful!") return
            redirect('home')
else:
form = UserCreationForm()
```

B. Sessions

Used for user authentication and the shopping cart storage.

```
def add_to_cart(request, product_id):
```

```
product = get_object_or_404(Product, id=product_id)


cart = request.session.get('cart') if

    not cart:

cart = {}


product_id = str(product_id) if

    product_id in cart:

        cart[product_id] += 1

    else:

cart[product_id] = 1


request.session['cart'] = cart

request.session.modified = True# Ensures session saves


messages.success(request, f"{product.name} added to cart successfully!")



return redirect('products')
```

3. Templating Concepts

Demonstrates **Template Inheritance** and **Dynamic Logic**.

```
<!-- templates/shop/base.html -->
```

```
{% block content %}{% endblock %}
```

```
<!-- templates/shop/home.html -->

{% extends 'store/base.html' %}

{% block content %}

<h2>Welcome, {{ user.username|default:"Guest" }}</h2>

{% endblock %}
```

4. **Database Integration: Relational, CRUD, & ORM**

- **Relational:** Uses SQLite (Tables with Relationships).
- **ORM:** Performs queries without manual SQL.
- **CRUD (Read):** `***products = Product.objects.all()***` fetches all rows from the database.

5. **Authentication & Authorization**

Uses Django's built-in `User` model and `login_required` decorator.

```
# store/views.py

from django.contrib.auth.decorators import login_required

@login_required

def checkout(request):
```

6. **Cookies & Sessions**

A. Cookies are stored in the user's browser and are used to store information about the user(like last login username).

```
# store/views.py (Inside login_user)
```

```
response = redirect('home')

response.set_cookie('last_login_user', user.username) # MANUAL COOKIE SET
```

B. **Sessions**

Sessions are stored on the server and are used to store information about the user(user authentication , cart-items).

```
def add_to_cart(request, product_id):

product = get_object_or_404(Product, id=product_id) cart

    = request.session.get('cart')

if not cart:

cart = {}


product_id = str(product_id) if

    product_id in cart:

        cart[product_id] += 1

    else:

        cart[product_id] = 1

    request.session['cart'] = cart

request.session.modified = True# Ensures session saves
```

7. **Middleware & Security**

- **Security:** Built-in `CsrfViewMiddleware` protects every POST form from cross- site attacks.

```
<form method="POST">
```

```
{% csrf_token %}

{{ form.as_p }}

<button type="submit" class="btn" style="width: 100%;">Login</button>

</form>
```

- **Built-in Middleware :**

  - `AuthenticationMiddleware` checks if the user is authenticated.

```
@login_required

def checkout(request):
```

  - `SessionMiddleware` handles session management.

```
# in add_to_cart view

cart = request.session.get('cart')
```

- **Middleware for Logging:** Custom `RequestLoggingMiddleware` tracks server performance.

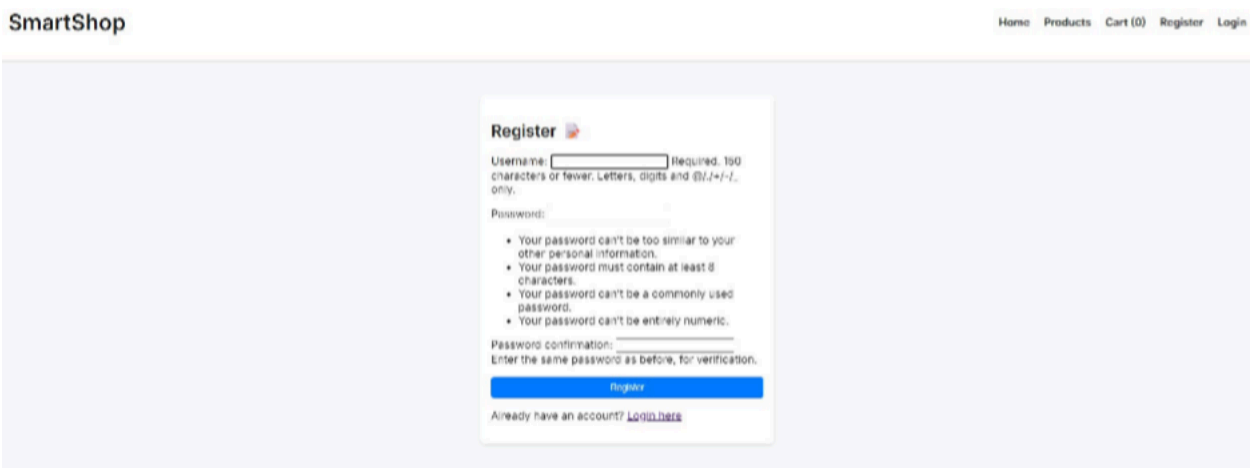
```
# in store/middleware.py

def RequestLoggingMiddleware(get_response):
```

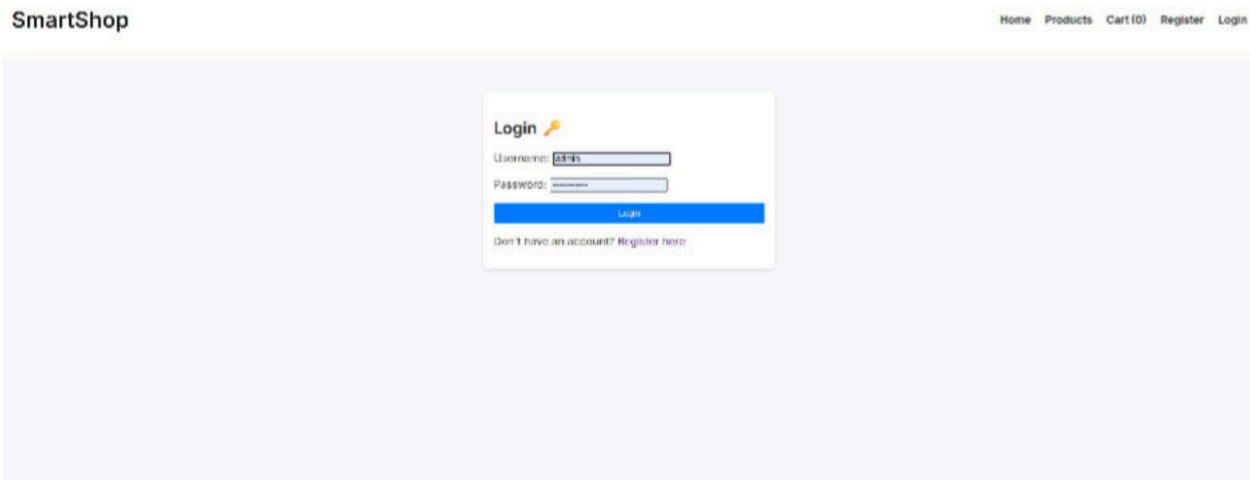
```
# in add_to_cart

product = get_object_or_404(Product, id=product_id)
```

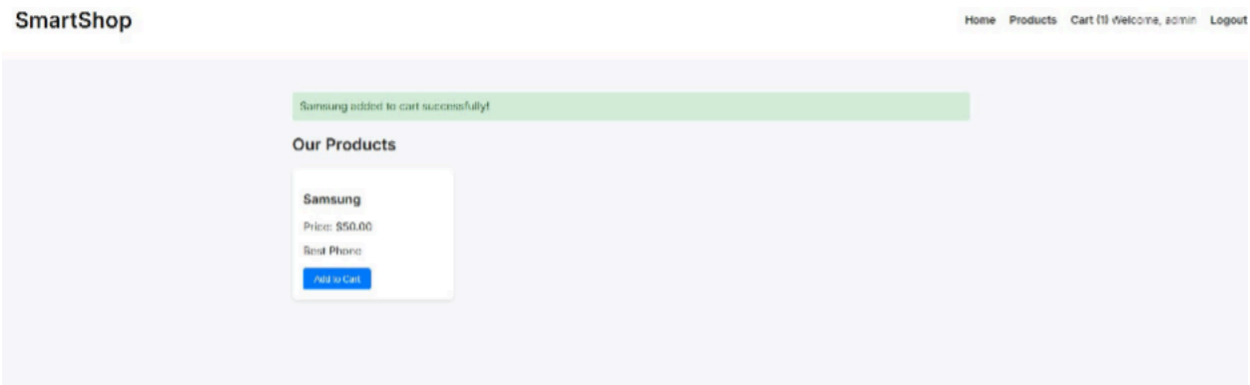
- **Error Handling:** `get_object_or_404` prevents server crashes by failing graceful

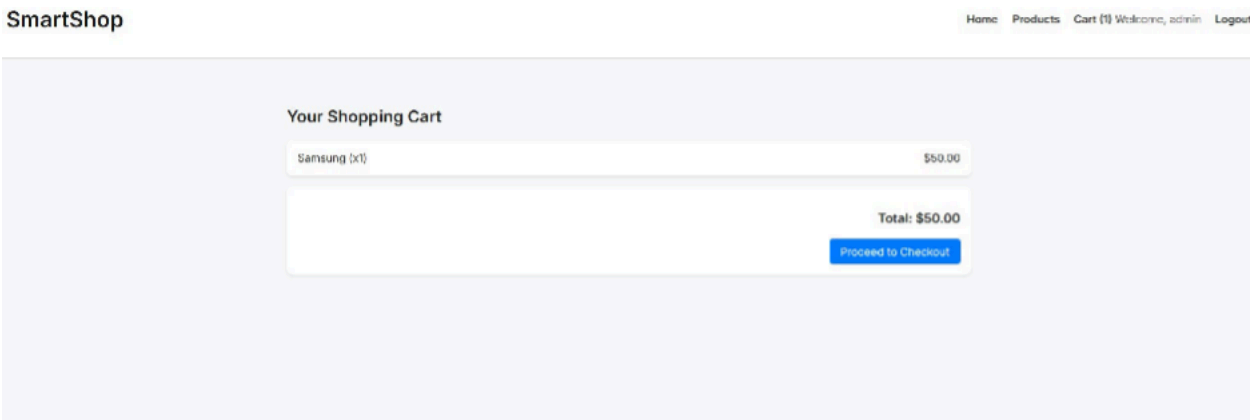
5. Output:

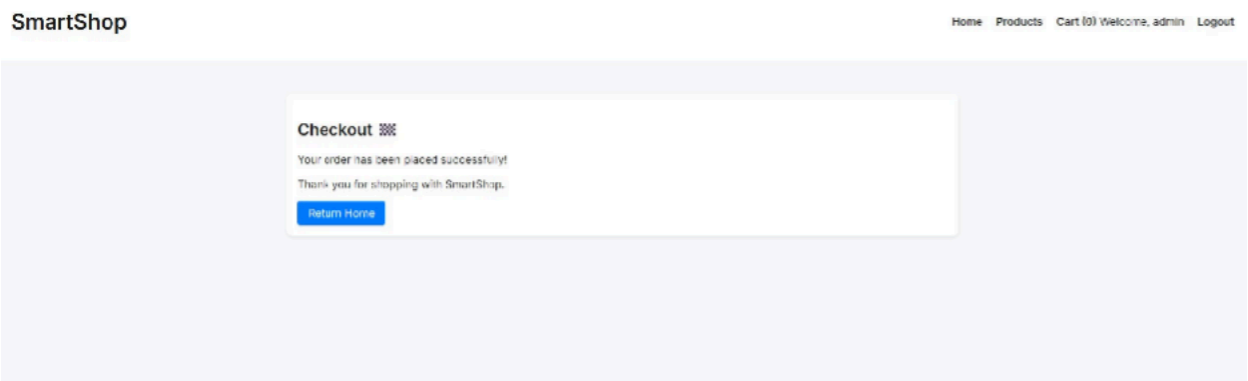
SmartShop
Home
Products
Cart (0)
Register
Login
Welcome to SmartShop!
View Our Products


SmartShop
Home
Products
Cart (0)
Register
Login
Register
Username:
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.
Password:
Your password can't be too similar to your other personal information.
Your password must contain at least 8 characters.
Your password can't be a commonly used password.
Your password can't be entirely numeric.
Password confirmation:
Enter the same password as before, for verification.
Register
Already have an account? Login here


SmartShop
Home Products Cart (0) Register Login
Login
Username: admin
Password: ••••••••••
Login
Don't have an account? Register here


SmartShop
Home Products Cart (1) Welcome, admin Logout
Samsung added to cart successfully!
Our Products
Samsung
Price: $50.00
Best Phone
Add to Cart


SmartShop
Home Products Cart (1) Welcome, admin Logout
Your Shopping Cart
Samsung (x1) $50.00
Total: $50.00
Proceed to Checkout


SmartShop
Home Products Cart (0) Welcome, admin Logout
Checkout
Your order has been placed successfully!
Thank you for shopping with SmartShop.
Return Home